

How Abilities in Large Language Models are Affected by Supervised Fine-tuning Data Composition

Guanting Dong*, Hongyi Yuan*, Keming Lu, Chengpeng Li*, Mingfeng Xue
Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, Jingren Zhou

Alibaba Group

{dongguanting.dgt,yuanzheng.yuanzhen,ericzhou.zc}@alibaba-inc.com

Abstract

Large language models (LLMs) with enormous pre-training tokens and parameters emerge diverse abilities, including math reasoning, code generation, and instruction following. These abilities are further enhanced by supervised fine-tuning (SFT). While the open-source community has explored ad-hoc SFT for enhancing individual capabilities, proprietary LLMs exhibit versatility across various skills. Therefore, understanding the facilitation of multiple abilities via SFT is paramount. In this study, we specifically focus on the interplay of data composition between mathematical reasoning, code generation, and general human-aligning abilities during SFT. We propose four intriguing research questions to explore the association between model performance and various factors including data amount, composition ratio, model size and SFT strategies. Our experiments reveal that distinct capabilities scale differently and larger models generally show superior performance with same amount of data. Mathematical reasoning and code generation consistently improve with increasing data amount, whereas general abilities plateau after roughly a thousand samples. Moreover, we observe data composition appears to enhance various abilities under limited data conditions, yet can lead to performance conflicts when data is plentiful. Our findings also suggest the amount of composition data influences performance more than the composition ratio. In analysis of SFT strategies, we find that sequentially learning multiple skills risks catastrophic forgetting. Our proposed **Dual-stage Mixed Fine-tuning (DMT) strategy** offers a promising solution to learn multiple abilities with different scaling patterns.

1 Introduction

Recent research has demonstrated the remarkable and versatile proficiency of large language models (LLMs) in dealing with a variety of real-world

tasks expressed in natural languages (Ouyang et al., 2022a; Anil et al., 2023; OpenAI, 2023; Luo et al., 2023a), especially Information Extraction (IE) (Lu et al., 2022; Xu et al., 2023b; Zhao et al., 2023; Cheng et al., 2023d; Wang et al., 2023b; Zhang et al., 2024b; Li et al., 2024), Information Retrieval (IR) (Zhu et al., 2024; Liu et al., 2024c) and Spoken Language Understanding (SLU) (Hosciłowicz et al., 2024; Yin et al., 2024; Cheng et al., 2023a, 2024; Dong et al., 2023a). Among the tasks, LLMs especially emerge with three outstanding abilities in reasoning (Cobbe et al., 2021; Wei et al., 2022), coding (Chen et al., 2021), and aligning general human intentions (Ouyang et al., 2022a), which have drawn much attention from the LLM research community. In order to further incentivize such abilities, it necessitates supervised fine-tuning (SFT) stages on annotated task data.

However, existing research has mostly conducted separate SFT investigations on each of the three tasks, where reasoning and coding abilities require SFT on in-domain human-annotated or augmented data (Yuan et al., 2023b; Luo et al., 2023b; Yu et al., 2024) while diverse and complex human instructions are applauded for aligning human intentions (Wang et al., 2023d; Taori et al., 2023; Cheng et al., 2023c; Xu et al., 2023a; Zhou et al., 2023a; Wang et al., 2023a; Lu et al., 2023). As shown by the strong performance of proprietary LLMs such as GPT-4 (OpenAI, 2023) and Claude, LLMs have the potential to master all the tasks in one model. Therefore, it is of paramount importance to investigate the versatile performance of SFT with composite task data, and understanding and addressing the challenges posed by the data composition problem in the SFT stage is crucial for further enhancing the capabilities of LLMs in a comprehensive manner.

In essence, the tasks of reasoning, coding, and aligning human intentions are of different characteristics. Reasoning and coding tasks require ad-

* Work done during internships at Alibaba Group.

hoc abilities of complex and detailed logic in decomposing task instructions and dealing with non-linguistic and symbolic features (Chen et al., 2021; Huang and Chang, 2023), whereas aligning human intentions requires versatility and understanding obscure intentions expressed in human instructions (Lu et al., 2023). Given the fundamental difference among the tasks, multi-task learning with composite data fine-tuning for small-scaled pre-trained language models is prone to catastrophic forgetting (De Lange et al., 2022), hindering the fine-tuned performance of one model on separate tasks. Many efforts have been made to compensate for the phenomenon (Liang et al., 2021; Xu et al., 2021; Yuan et al., 2023a). There has also been research discovering that scaling up the pre-trained language model scale and the fine-tuning data scale are beneficial for zero-shot out-of-domain generalization on various linguistic tasks while leaving out the assessment of in-domain performance (Sanh et al., 2022; Chung et al., 2022a; Longpre et al., 2023). Given the increased capacity of LLMs, the multi-task performance by SFT on composite data of essentially different downstream tasks is less studied. Understanding the SFT performance with composite data and corresponding scaling patterns is of great utility in practice.

In this study, we focus on the data composition problem among **mathematical reasoning**, **code generation**, and **general human-aligning abilities** in SFT. We aim to comprehensively investigate the relationship between model performance and different factors including data amount, data composition ratio, model scales, and SFT training strategies. We also investigate how the relationship varies under different scales. Specifically, we focus on the following four research questions:

1. *How do math reasoning, coding, and general abilities scale with SFT data amounts?*
2. *Are there performance conflicts when combining these three abilities in SFT?*
3. *What are the key factors that induce the performance conflicts?*
4. *What are the impacts of different SFT strategies for composite data?*

To answer these questions, we conduct experiments on three benchmarks, which are GSM8K (Cobbe et al., 2021) for mathematical reasoning, HumanEval (Chen et al., 2021) for coding, and MT-Bench (Zheng et al., 2023) for general human alignment. We fine-tune LLMs on the related training data to activate these abilities. Furthermore, we

conduct extensive analysis regarding model parameter scales ranging from LLaMA 7B to 33B (Touvron et al., 2023) and explore four different SFT strategies shown in Figure 1: multi-task learning, sequential training, mixed sequential training, and dual-stage mixing fine-tuning (DMT), providing empirical guidance for learning a versatile LLM with composite SFT. The key findings of this paper can be summarized as follows:

- Different SFT abilities exhibit distinct scaling patterns, while larger models show better performances with the same data amount generally.
- Compared to single ability learning, multi-task learning multiple abilities exhibits improvement in low-resource and decline in high-resource. Additionally, as the model size increases, there is a greater performance gain in low-resource settings for math and general abilities.
- Data amounts directly influence each ability, while the data ratio is insignificant.
- Multi-task learning lead to conflicts, while sequential training results in catastrophic forgetting. Our proposed DMT effectively alleviates both performance conflicts and catastrophic forgetting in the SFT phrase, achieving a balance between general and specialized abilities.

2 Related Works

Supervised Fine-Tuning of Large Language Models Large Language Models (LLMs) have shown notable zero-shot performance in various domains (Brown et al., 2020; Wu et al., 2021; Hou et al., 2024; Dong et al., 2023b; Zhou et al., 2024; Wu et al., 2023; Song et al., 2023), prompting further development to push the boundaries of these models. To delve deeper to their potential, LLMs are subjected to a Supervised Fine-Tuning (SFT) phase, enhancing their ability to solve tasks and align better with human instructions. Here, we extend the conventional definition of SFT to include various forms of sequence-to-sequence fine-tuning, such as fine-tuning for human alignment, instruction following, and domain-specific task optimization (Zhou et al., 2023b; Yuan et al., 2023c; Cheng et al., 2023b; Zhang et al., 2024a).

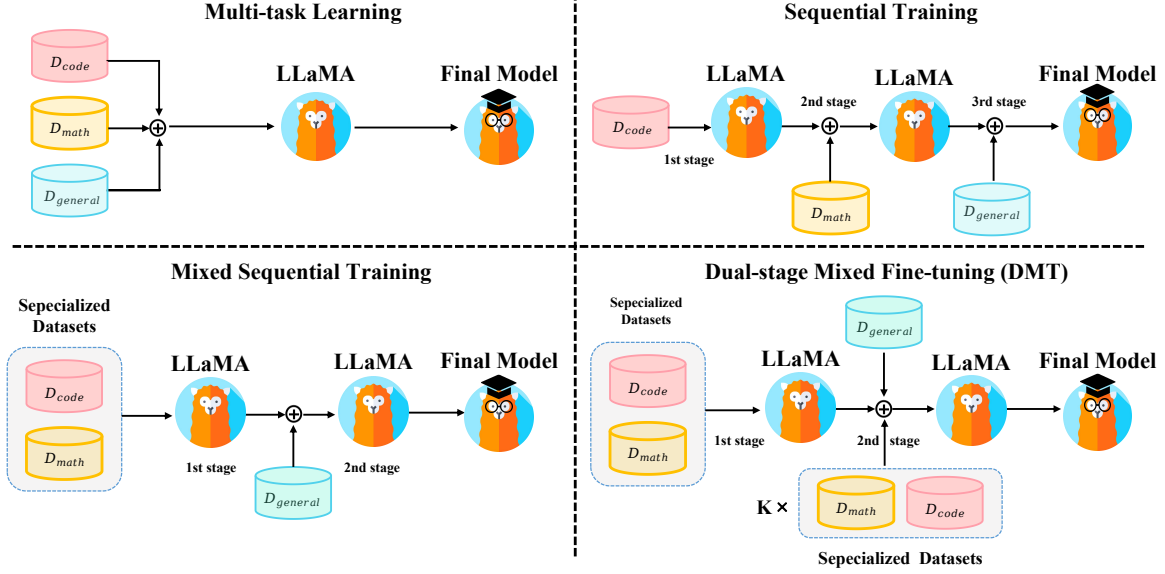


Figure 1: The illustration of four different training strategies in this paper.

Recent research has delved into multi-task instruction fine-tuning of pre-trained LLMs to bolster their zero-shot performance across numerous downstream NLP tasks (Sanh et al., 2022). In an effort to encompass existing NLP tasks comprehensively, Chung et al.; Longpre et al. curated the expansive FLAN dataset specifically for instruction-based fine-tuning. LLMs, both open-source (Chung et al., 2022b) and proprietary (Singhal et al., 2022), fine-tuned with FLAN, have demonstrated enhanced zero-shot performance on a variety of unseen tasks.

While research has probed into the generalization capabilities of LLMs within out-of-distribution domains (Liu et al., 2024a; Yuan et al., 2024; Wang et al., 2024a), the effects of multi-task training on in-domain performance remain under-explored. With the ascent of proprietary models like ChatGPT, the focus on SFT for aligning LLMs with human intent has intensified (Ouyang et al., 2022b). Moving away from crowd-sourced SFT data, recent initiatives have generated SFT datasets from user logs within proprietary LLM platforms (Chiang et al., 2023; Wang et al., 2023a), employing the models themselves to assist in the data generation process (Wang et al., 2023d; Taori et al., 2023; Cheng et al., 2023d; Lei et al., 2023; Xu et al., 2023a; Xue et al., 2023). Additionally, methods to improve SFT data quality have been proposed, targeting more accurate alignment with human interactions (Zhou et al., 2023a; Wang et al., 2023c; Lu et al., 2023; Liu et al., 2024b).

Furthermore, SFT has proven beneficial for

LLMs in specialized areas such as mathematical reasoning (Cobbe et al., 2021; Hendrycks et al., 2021; Yuan et al., 2023b; Chen et al., 2024; Yue et al., 2023; Gou et al., 2024; Li et al., 2023; Yue et al., 2024) and code generation tasks (Chaudhary, 2023; Luo et al., 2023b; Wang et al., 2024b; Wei et al., 2023). Taking advantage of their advanced interactive capabilities, some researchers have leveraged supervised fine-tuned LLMs to compose commands that interface with external tools, thus enhancing the handling of assorted downstream applications (Shen et al., 2023; Yao et al., 2023b,a; Song et al., 2024; Fu et al., 2024). This paper examines the SFT performance using composite datasets, considering different model sizes and data amounts.

Scaling Laws in Large Language Models The exceptional performance of LLMs comes from scaling up model sizes, data amounts, and computational costs to massive scales. Therefore, it is crucial to explore the model performance across an exponential range of scales. Many endeavors have been made to discuss the scaling laws for pre-training (Anil et al., 2023; Hoffmann et al., 2022), transfer learning (Chronopoulou et al., 2019), preference modeling (Gao et al., 2022) and mathematical reasoning (Yuan et al., 2023b). In this paper, we also explore the SFT performance with composite data from the perspective of different scales of model sizes and data amounts.

3 Experiments

We have SFT datasets $\{D_1, D_2, \dots, D_k\}$ where each $D_i = \{q_{i,j}, r_{i,j}\}_j$ contains queries and responses from one source. We consider each SFT dataset to correspond to one ability and we also have k in-domain metrics to measure them. We investigate the performances of in-domain metrics with different dataset compositions ($D \subset \cup_{1 \leq i \leq k} D_i$) and training strategies on different sizes of LLMs.

3.1 Experiment Setup

We collect three SFT datasets $\{D_1, D_2, D_3\}$ including GSM8K RFT (Yuan et al., 2023b), Code Alpaca (Chaudhary, 2023), and ShareGPT (Chiang et al., 2023) to represent math reasoning, coding, and general human-aligning ability SFT dataset respectively. We will integrate a new SFT dataset D by these three datasets to investigate how data composition affects the model performances. We use GSM8K test set (Cobbe et al., 2021), HumanEval (Chen et al., 2021), and MT-Bench (Zheng et al., 2023) to measure abilities including math reasoning, coding, and general human-aligning. We use LLaMA (Touvron et al., 2023) series as our pre-trained language models and use FastChat framework (Zheng et al., 2023) for fine-tuning. We fine-tune models with 3 epochs and a peak of $2e-5$ learning rate. The batch size during SFT is 16. More details about SFT datasets, evaluation metrics, implementations and Training FLOPs can be found in Appendix A, B, C and D.

3.2 RQ1. Individual Ability Performance vs. Data Amount

The instruction following ability can be activated via SFT on datasets like ShareGPT which contain around 100 thousand samples. However, (Zhou et al., 2023a) demonstrates that strong base models can achieve human alignment with just 1000 samples. Specialized abilities such as math reasoning require a large amount of data (Cobbe et al., 2021; Yuan et al., 2023b), unlike general abilities. Therefore, it is crucial to investigate how each ability improves as the data amount increases.

Experimental Design: We conduct SFT on LLaMA of various sizes using $\{1, 1/4, 1/16, 1/64, 1/256\}$ proportions of the training set obtained from GSM8K RFT, Code Alpaca, and ShareGPT separately. This allowed us to evaluate each ability with various data sizes and model sizes.

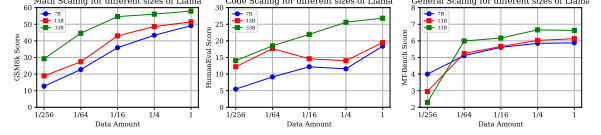


Figure 2: The scaling curve of different sizes of LLaMA in three individual domains.

Results and Analysis. Figure 2 shows the individual data scaling curves for different abilities after SFT. We find that: **Different abilities exhibit different scaling curves.** To be more specific, mathematical reasoning capability shows a positive correlation with the data amount across various model sizes which is consistent with (Yuan et al., 2023b). Similarly, general human-aligning ability demonstrates an almost monotonically increasing scaling curve. However, it is noteworthy that general ability emerges with only around 1k data samples (ranging from 1/256 to 1/64), and after reaching a certain threshold (1/64), their performances improve slowly. This further supports (Zhou et al., 2023a), indicating that a small amount of high-quality SFT data is possible for the emergence of general human-aligning ability in LLMs. On the other hand, code ability exhibits an irregular scaling curve when the model’s parameter count is small (7B & 13B). However, when the parameter count increases to 33B, its coding performance shows an approximately log-linear trend with the data amount. One possible explanation is that Code Alpaca and the samples in HumanEval have different distributions. Larger models can capture shared knowledge across code data distributions in the in-domain samples, which enables them to exhibit some level of generalization to out-of-distribution (OOD) samples. Another observation is **larger models show better performances with the same data amount generally.** The outlier is with very little data (1/256), smaller models may outperform larger models. If there is enough data, larger models have stable better performances.

3.3 RQ2. Performance Difference vs. Mixed Data Amount

We should deliver a versatile model that requires us to mix various SFT datasets and apply SFT. We want to ask how each ability varies due to SFT dataset mixtures. We investigate it with different amounts of mixed data and compare them with individual ability performance.

Experimental Design: For the individual source

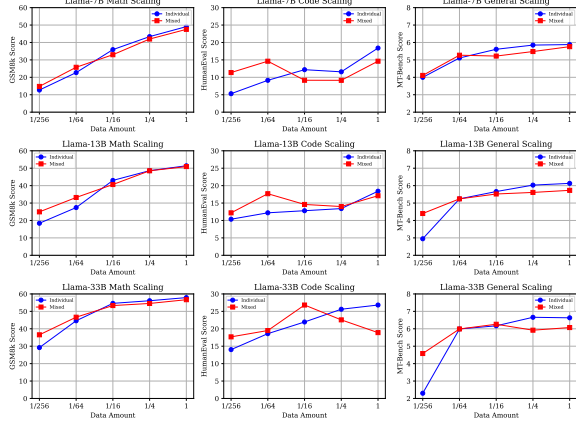


Figure 3: Comparative experiments between mix domains and individual domains for LLaMA.

setting, consistent with the setup in RQ1, we performed fine-tuning on LLaMA models of different sizes using $\{1, 1/4, 1/16, 1/64, 1/256\}$ amounts of training data from GSM8K, Code Alpaca, and ShareGPT separately. For the mixed source setting, we sampled $\{1, 1/4, 1/16, 1/64, 1/256\}$ amounts of training data from GSM8K, Code Alpaca, and ShareGPT, and directly mixed them according to the corresponding proportions. In this way, we constructed datasets with fixed proportions of different ability domains, while varying the total data amount. These datasets are then used for fine-tuning the LLaMA models ¹.

Results and Analysis. Figure 3 presents results of LLaMA of different sizes on three benchmarks under the individual source and mixed source settings. The following observations are made: **Abilities are improved with low-resource and are decreased with high-resource compared to individual source abilities.** In the case of LLaMA-7B, compared to the data scaling curve of the individual source setting, the models fine-tuned with mixed source data consistently demonstrated performance conflicts among the three ability domains at high resources (100%). However, as the data volume decreased, a turning point in performance is observed between the two settings in the data range of 1/64 to 1/16. Notably, the models fine-tuned with mixed source data exhibited performance gains at low resources (1/256), indicating that SFT data from different sources benefit each other in a low-resource setting. However, when there is enough data, data from other sources could be viewed as noise for

¹We also conduct "Equal Data Amount VS. Equal Data Proportion" experiments in Appendix H

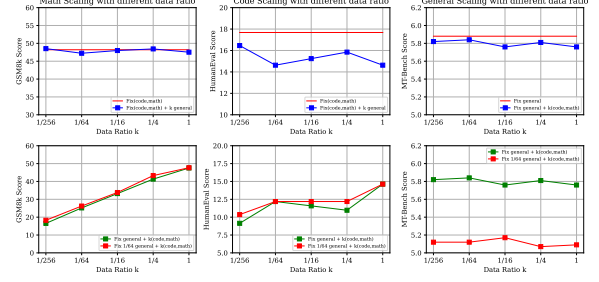


Figure 4: Different data ratio (k) between specific abilities and general abilities on three benchmarks.

in-domain generalization. **As the model size increases, the performance gain in low-resource settings also increases for math and general abilities.** In the case of the 13B and 33B models, it is obvious that the scaling curve for the mix source setting follows a similar trend observed in previous analyses, with the presence of performance intersection points as the data volume scales. However, a crucial distinction arises, whereby larger models exhibit more pronounced performance gains under low resources as the size of model parameters increases. The outlier is the LLaMA-7B (code only, 1/256). A possible reason is the introduction of a small amount of unseen code data easily disrupts the original code ability of the pretrained model, as supported by its low HumanEval score (less than 6). In conclusion, our finding implies that larger language models excel in acquiring general and specialized abilities from diverse data sources under low-resource conditions ².

3.4 RQ3. Performance Difference vs. Data Composition Ratio

We observe ability conflicts in high-resource settings, and we want to investigate the reasons why the conflicts occur. Two possible factors are the **data amount** of other abilities is too high or the **data ratio** of other abilities is too high. Here we conduct experiments to investigate the data ratio factor.

Experimental Design: We consider coding and mathematics as a combined specialized data source, and the ShareGPT as the general data source. We designed three setups as follows which control the amount of one source of data and vary the ratio between general and specialized data.

1. Fixed general data, scaling specialized data:

²To validate the generalizability of our conclusions, we further conduct the more experiments on **World Knowledge**, **Language Understanding** and **Translation** in Appendix E.

We use a full training set of ShareGPT and sampled different proportions $\{1, 1/4, 1/16, 1/64, 1/256\}$ of GSM8K RFT and Code Alpaca as a mixture.

2. Fixed specialized data, scaling general data:

We use a full training set of GSM8K RFT and Code Alpaca and sample different proportions of ShareGPT as a mixture.

3. Fixed 1/64 general data, scaling specialized data: Motivated by LIMA’s setup (Zhou et al., 2023a), we used a 1/64 ShareGPT set (about 1500 examples) and sampled different proportions of GSM8K RFT and Code Alpaca as a mixture.

Results and Analysis. Q1: Does the performance of the model vary with different ratios of general and specialized data? As illustrated in the top three graphs of Figure 4, we conduct ablation studies of the data ratio (k) between specialized and general abilities. To be noticed ratio is normalized by data amount, for example, $k = 1$ means $\frac{\text{specialized use data amount}}{\text{general use data amount}} = \frac{\text{specialized all data amount}}{\text{general all data amount}}$. We utilize a fixed specialized data setting (directly mixing 100% code & math data for training) and a fixed general data setting (100% general data for training) as the baseline and observe:

(1) With the increase in the ratio of general data from 1/256 to 1/1, *Fixed specialized data, scaling general data* setup exhibits similar performance to the setup that *Fixed specialized abilities* in terms of math reasoning. This suggests that variations in the data ratio k have minimal impact on math ability. We consider the reason that math and general abilities are non-conflict since they are too different in the semantic space. However, when considering HumanEval, the *Fixed specialized data, scaling general data* setup displays noticeable fluctuations compared to the baseline. We attribute this to the inclusion of a certain proportion of code data in ShareGPT. Due to the differences in data format and distribution, the presence of similar data features exacerbates the performance conflicts between abilities when the data ratio k increases. Further analysis of the distribution of different abilities is discussed in Section 4.1.

(2) With the increase in the ratio of specialized data from 1/256 to 1/1, the setup that *Fixed general data, scaling specialized data* displayed no significant performance changes compared to the baseline. This echoes our hypothesis that when **there are significant differences in task formats and data distributions between different SFT**

abilities, the impact of data ratio is minimal. However, when **there is some degree of similarities, the data ratio can lead to noticeable performance fluctuations.**

Q2: Under extremely limited general data resources, does the ratio of specialized data have an impact on the model’s performance? We further explore the impact of different ratios of specialized data when the model has just acquired a certain level of general human-aligning ability ($k = 1/64$). The bottom 3 graphs of Figure 4 present comparative experiments between two settings. We observe that regardless of whether the data amount for general capabilities is abundant ($k = 1$) or scarce ($k = 1/64$), the performance on MT-Bench shows no significant fluctuations with varying proportions of specialized data. Furthermore, in mathematical reasoning, 1/64 general data setup exhibited a scaling trend that is almost identical to the full general data setup. However, for coding ability, with the same amount of code data and different ratios, code abilities are different in the two settings. We still consider the reason is code data are partly related to ShareGPT data and cause the performance difference and provide an analysis in Discussion 4.2.

3.5 RQ4. Performance Difference vs. Training Strategies

We could feed these SFT datasets into models with different training strategies. In this section, We experiment with these settings and investigate how they influence each ability’s performance.

Experimental Design: Firstly, we introduce three kinds of naive training strategies as follows:

1. Multi-task learning: We directly mix different SFT data sources $D = \cup_{1 \leq i \leq k} D_i$ and applying SFT. If we view each data source as a different task, this can be viewed as multi-task learning.

2. Sequential Training: We sequentially apply SFT on each dataset. Specifically, we sequentially trained on coding, math reasoning, and the general ability dataset. Since the general ability is the most important one for human alignment, we put ShareGPT as our last dataset.

3. Mixed Sequential Training: We apply multi-task learning on specialized datasets(code, math) first and apply SFT on the general ability dataset. These three approaches are presented in Figure 1.

Results and Analysis: Table 1 presents performances under different training strategies in terms

of mathematical reasoning, code generation, and general human-aligning ability. Multi-task learning preserves specialized abilities among these strategies while hurting the general ability most among them. Sequential training and mixed sequential training preserve general ability while losing too many specialized abilities. The observed outcome is in accordance with expectations, as during the final fine-tuning phase, the mixed sequential training strategy remains unaffected by specialized data, thereby effectively preserving its generalization capability. However, an inherent drawback of multi-stage training is the occurrence of catastrophic forgetting of prior knowledge, which motivates us to further explore methods that can alleviate catastrophic forgetting of specialized abilities while maximizing the preservation of general capability.

4. Dual-stage Mixed Fine-tuning (DMT):

Based on our observation from RQ1 to RQ4, we propose a new training strategy that can reduce the ability conflict during multi-task learning and relieve the issue of catastrophic forgetting during sequential training. From RQ1, the model needs large data amounts to activate specialized abilities. From RQ2, multi-task learning with all amounts of specialized data and general data will hurt each ability. From RQ3, a small amount of specialized data will not affect the general ability performance. From RQ4, (mixed) sequential training forgets specialized abilities. So the model needs to learn large amounts of specialized data and should not forget them during learning general ability. A natural choice is to learn full amounts of specialized data first and add a small amount of specialized data to general data during the last stage of sequential training to prevent forgetting. As shown in Figure 1, we first apply SFT on the specialized dataset which is same as the first stage of the mixed sequential training strategy. For the second stage, we perform SFT with a mixed data source comprising a combination of the general data and varying proportions k (1, 1/2, 1/4, 1/8, 1/16, 1/32) of code and math data. Adding code and math data in the second stage helps models to recall the specialized ability. The results of DMT ($k = 1/256$) are presented in Table 1 and the detailed scaling analysis of proportion k can be found in the discussion.

Model Accuracy vs. DMT Strategies. In Table 1, LLaMA-7B with DMT ($k = 1/256$) strategy perform significant improvement in mathematical reasoning (32.6 to 41.92) and code generation (15.24 to 17.68) compared to the mixed sequential training

strategy, which indicates a significant alleviating effect of mixing specialized capability data in the last fine-tuning stage on catastrophic forgetting. Surprisingly, DMT ($k = 1/256$) even exhibits a slight improvement on MT-Bench, further highlighting its ability to alleviate catastrophic forgetting while effectively preserving general capability.

Regarding the 13B and 33B models, DMT ($k = 1/256$) demonstrates noticeable alleviation of catastrophic forgetting in mathematical reasoning (13B: 40.48 to 46.47 / 33B: 44.24 to 56.36) and code generation (13B: 18.3 to 19.5 / 33B: 24.4 to 25.5) compared to the mixed sequential training strategy. Additionally, it significantly retains its general capability (13B: 5.93 to 6.03 / 33B 6.43 to 6.69). Therefore, these results serve as additional validation of the efficacy of DMT in mitigating catastrophic forgetting while maintaining general capability³.

4 Discussion

4.1 Visualization of Different SFT Abilities

In the aforementioned analysis of data composition, we observed a significant performance degradation when different data sources are directly mixed. In this section, our aim is to explore the potential mutual influence of semantic representation distributions among different data sources. Specifically, we randomly sampled 100 queries from CodeAlpaca, GSM8k RFT, and ShareGPT datasets and extracted the hidden layer representations located in the Middle layer (15th) of the model. Subsequently, we employed the t-SNE toolkit (Van der Maaten and Hinton, 2008) to visualize the representations of the three types of capabilities. The results in Figure 5 illustrate a notable collapse phenomenon in the semantic representations of both the original LLaMA-13b and LLaMA-13b with DMT ($k=1/256$). While both models exhibit a certain level of separation in the mathematical data representations, there remains a certain degree of overlap between the representations of code and general samples. In Appendix G, we further discuss the visualization of semantic spaces at different layers of LLaMA 7B & 13B.

³To verify the effectiveness of DMT strategy on relatively OOD benchmarks, we further evaluate it on MBPP and MATH in Appendix F.

Methods	LLaMA -7B			LLaMA -13B			LLaMA -33B		
	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench
<i>Individual domain</i>									
General only	11.10	10.42	5.88	14.02	16.40	6.13	26.06	24.30	6.63
Math only	49.10	6.71	2.53	51.40	12.8	2.54	57.91	15.5	3.18
Code only	4.51	18.40	4.30	5.15	17.1	3.53	6.06	26.82	4.18
<i>Different Training Strategies</i>									
Multi-task learning	47.53	14.63	5.76	50.94	<u>19.50</u>	5.73	56.69	18.9	6.07
Sequential Training	31.39	<u>15.85</u>	5.72	39.12	20.12	<u>5.93</u>	47.27	<u>24.80</u>	6.73
Mixed Sequential Training	32.60	15.24	<u>6.02</u>	40.48	18.30	<u>5.93</u>	44.24	24.4	6.43
DMT($k=1/256$)	<u>41.92</u>	17.68	6.08	<u>46.47</u>	<u>19.50</u>	6.03	<u>56.36</u>	25.00	6.73

Table 1: The results of LLaMA-7B, 13B, 33B under different training strategies on three benchmarks. The top two results across different strategies are marked with **bold** and underlined.

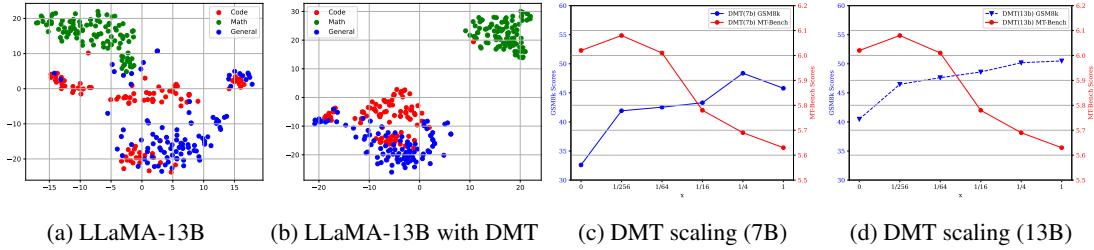


Figure 5: The left two figures show the t-SNE plots of LLaMA-13B and LLaMA-13B with the DMT strategy. The two right figures show the performance scaling of LLaMA-7B & 13B with DMT under different k values.

4.2 Ablation of the Specialized Domains in ShareGPT

In RQ2, we observe using mixed data sources resulted in improved abilities under low-resource conditions but diminished abilities under high-resource conditions when compared to single data sources. However, the presence of coding and mathematical samples within the ShareGPT introduces uncertainty regarding whether the performance gain under low resources is solely attributed to these specific coding & mathematical data or other orthogonal samples in the general dataset (e.g., translation or extraction). Hence, the objective of this section is to investigate whether the conclusions drawn in Section 3.3 remain valid after removing the code and math samples within ShareGPT.

Experimental Design: We employed an open-set tagger InsTag (Lu et al., 2023) to annotate samples in ShareGPT. To filter out data related to coding and mathematical abilities, we conduct regular expression matching to eliminate instances where the tags contain keywords “code” or “math”. Finally, we obtain a ShareGPT dataset devoid of any code or math-related information (reducing from 86K to 63K). In alignment with the settings in Section 3.3, we sampled different proportions of training data (1, 1/4, 1/16, 1/64, 1/256) from GSM8K,

Code Alpaca, and the modified ShareGPT dataset (without code math). These samples were directly mixed according to the corresponding proportions. Subsequently, the LLaMA models were fine-tuned by using this mixed dataset.

Results and Analysis. Figure 6 shows the results of our experiment. Removing the code and math from ShareGPT not only mitigates the performance conflicts among different abilities to some extent under high-resource conditions but also maintains stable gains in low-resource settings. We propose that the potential reason behind these findings lies in the differences in the distribution of code and math data between ShareGPT, CodeAlpaca, and GSM8K RFT datasets. This distribution gap introduces an extra noise during the SFT phrase, while its removal enables the model to better generalize coding and mathematical abilities. Furthermore, in low-resource scenarios, this phenomenon indicates that the code and math samples in ShareGPT are not the key factor contributing to performance improvements, but rather the diversity and variability of the data (Longpre et al., 2023). In summary, the presence of code math data within ShareGPT does not emerge as a key factor impacting the performance gains identified in Section 3.3, highlighting the generalization of our conclusions.

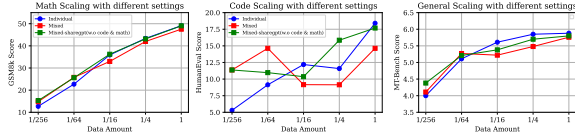


Figure 6: The scaling curve after ablating code and math-related samples from ShareGPT.

4.3 Specialized Data Amount in DMT

We investigate how different values of k influence model performance and results shown in Figure 5. When we adjust k from 0 to $1/256$ ($k = 0$ is equal to mixed sequential training), the SFT models show significant improvements in both specialized ability and general human-aligning ability. On the contrary, as k increased from $1/4$ to 1 , the model exhibited a decline in general ability. We believe this is in line with the findings in RQ2, which concluded that high-resource settings lead to conflicts while low-resource settings lead to gains in mixed sources. Furthermore, as k increased from $1/256$ to $1/4$, we observe a linear inverse trend between general ability and specialized ability, especially an increase in general ability coincided with a decrease in specialized ability. This suggests k needs to be tuned based on specific requirements in order to achieve a balance between multiple abilities.

5 Conclusion

We explore the data composition in the SFT phase, focusing on mathematical reasoning, code generation, and general human-aligning abilities. We formulate four research questions to guide our investigation and analyze the scaling trends between different abilities and factors (e.g. data amount, data ratio, model parameters, and training strategies). Our findings reveal distinct scaling patterns among different abilities, with larger models demonstrating superior performance when trained with the same amount of data. Moreover, mixing data sources in the SFT phase improves performance in low-resource scenarios but diminishes in high-resource scenarios. Interestingly, the phenomenon of low-resource gain becomes more prominent as the model parameter size increases. Furthermore, our observations indicate that data amount directly influences performance conflicts, whereas the impact of data ratio is insignificant within our experimental setup. Finally, regarding the SFT strategies, we demonstrate our proposed DMT strategy effectively alleviates performance conflicts, offering a

promising solution to activate multiple abilities.

Limitations

Due to our use of the large language model LLaMA-33B, the extensive computational resources and time required for both training and inference may limit its applicability. The datasets used in this article are all open source, so there are no ethical or moral issues; However, inappropriate prompts and noisy training corpora can potentially lead to privacy and bias issues with LLMs. Furthermore, the evaluation benchmark MT-Bench relies on GPT-4 for scoring, which may result in some variability in the results, and these may not always align perfectly with human judgment standards. In this paper, we primarily focus on three SFT capabilities that are of great interest in the LLMs community, including mathematical reasoning, code generation, and general human-aligned ability. To verify the generality of our conclusions, we further explore three additional SFT capabilities in the appendix. Nevertheless, there are still many other SFT capabilities (such as creative generation) within the LLMs community that have data composition issues waiting to be explored by researchers, which will also be the focus of our future research efforts.

References

- Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Loubna Ben Allal, Niklas Muennighoff, Logesh Kumar Umapathi, Ben Lipkin, and Leandro von Werra. 2022. A framework for the evaluation of code generation models. <https://github.com/bigcode-project/bigcode-evaluation-harness>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess,

- Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. [Language models are few-shot learners](#).
- Mauro Cettolo, Jan Niehues, Sebastian Stüker, Luisa Bentivogli, and Marcello Federico. 2014. [Report on the 11th IWSLT evaluation campaign](#). In *Proceedings of the 11th International Workshop on Spoken Language Translation: Evaluation Campaign*, pages 2–17, Lake Tahoe, California.
- Sahil Chaudhary. 2023. Code alpaca: An instruction-following llama model for code generation. <https://github.com/sahil280114/codealpaca>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. [Evaluating large language models trained on code](#).
- Zhaorun Chen, Zhuokai Zhao, Zhihong Zhu, Ruiqi Zhang, Xiang Li, Bhiksha Raj, and Huaxiu Yao. 2024. Autoprml: Automating procedural supervision for multi-step reasoning via controllable question decomposition. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Xuxin Cheng, Bowen Cao, Qichen Ye, Zhihong Zhu, Hongxiang Li, and Yuexian Zou. 2023a. MI-lmcl: Mutual learning and large-margin contrastive learning for improving asr robustness in spoken language understanding. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 6492–6505.
- Xuxin Cheng, Qianqian Dong, Fengpeng Yue, Tom Ko, Mingxuan Wang, and Yuexian Zou. 2023b. M 3 st: Mix at three levels for speech translation. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.
- Xuxin Cheng, Zhihong Zhu, Bowen Cao, Qichen Ye, and Yuexian Zou. 2023c. Mrml: Modifying the reference via reinforcement learning for non-autoregressive joint multiple intent detection and slot filling. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10495–10505.
- Xuxin Cheng, Zhihong Zhu, Hongxiang Li, Yaowei Li, Xianwei Zhuang, and Yuexian Zou. 2024. Towards multi-intent spoken language understanding via hierarchical attention and optimal transport. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17844–17852.
- Xuxin Cheng, Zhihong Zhu, Wanshi Xu, Yaowei Li, Hongxiang Li, and Yuexian Zou. 2023d. Accelerating multiple intent detection and slot filling via targeted knowledge distillation. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. [Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality](#).
- Alexandra Chronopoulou, Christos Baziotis, and Alexandros Potamianos. 2019. An embarrassingly simple approach for transfer learning from pretrained language models. *arXiv preprint arXiv:1902.10547*.
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022a. [Scaling instruction-finetuned language models](#).
- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022b. [Scaling instruction-finetuned language models](#).
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2022. [A continual learning survey: Defying forgetting in classification tasks](#). *IEEE*

- Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385.
- Guanting Dong, Tingfeng Hui, Zhuoma GongQue, Jinxu Zhao, Daichi Guo, Gang Zhao, Keqing He, and Weiran Xu. 2023a. [Demonsf: A multi-task demonstration-based generative framework for noisy slot filling task](#).
- Guanting Dong, Jinxu Zhao, Tingfeng Hui, Daichi Guo, Wenlong Wan, Boqi Feng, Yueyan Qiu, Zhuoma Gongque, Keqing He, Zechen Wang, et al. 2023b. Revisit input perturbation problems for llms: A unified robustness evaluation framework for noisy slot filling task. *arXiv preprint arXiv:2310.06504*.
- Dayuan Fu, Jianzhao Huang, Siyuan Lu, Guanting Dong, Yejie Wang, Keqing He, and Weiran Xu. 2024. [Preact: Predicting future in react enhances agent’s planning ability](#).
- Leo Gao, John Schulman, and Jacob Hilton. 2022. [Scaling laws for reward model overoptimization](#).
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujia Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. [Tora: A tool-integrated reasoning agent for mathematical problem solving](#).
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. [Training compute-optimal large language models](#).
- Jakub Hoscilowicz, Pawel Pawlowski, Marcin Skorupa, Marcin Sowański, and Artur Janicki. 2024. [Large language models for expansion of spoken language understanding systems to new languages](#).
- Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*, pages 364–381. Springer.
- Jie Huang and Kevin Chen-Chuan Chang. 2023. [Towards reasoning in large language models: A survey](#). In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1049–1065, Toronto, Canada. Association for Computational Linguistics.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Shanglin Lei, Guanting Dong, Xiaoping Wang, Keheng Wang, and Sirui Wang. 2023. [Instructerc: Reforming emotion recognition in conversation with a retrieval multi-task llms framework](#).
- Chengpeng Li, Zheng Yuan, Hongyi Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. 2023. [Query and response augmentation cannot help out-of-domain math reasoning generalization](#).
- Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. 2024. [From matching to generation: A survey on generative information retrieval](#).
- Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. 2021. [R-drop: Regularized dropout for neural networks](#).
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*.
- Bo Liu, Liming Zhan, Zexin Lu, Yujie Feng, Lei Xue, and Xiao-Ming Wu. 2024a. [How good are llms at out-of-distribution detection?](#)
- Wei Liu, Weihao Zeng, Keqing He, Yong Jiang, and Junxian He. 2024b. [What makes good data for alignment? a comprehensive study of automatic data selection in instruction tuning](#).
- Zheng Liu, Yujia Zhou, Yutao Zhu, Jianxun Lian, Chaozhuo Li, Zhicheng Dou, Defu Lian, and Jian-Yun Nie. 2024c. [Information retrieval meets large language models](#). In *Companion Proceedings of the ACM on Web Conference 2024, WWW ’24*, page 1586–1589, New York, NY, USA. Association for Computing Machinery.
- Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, et al. 2023. The flan collection: Designing data and methods for effective instruction tuning. *arXiv preprint arXiv:2301.13688*.
- Keming Lu, Hongyi Yuan, Zheng Yuan, Runji Lin, Junyang Lin, Chuanqi Tan, and Chang Zhou. 2023. [# instag: Instruction tagging for diversity and complexity analysis](#). *arXiv preprint arXiv:2308.07074*.
- Yaojie Lu, Qing Liu, Dai Dai, Xinyan Xiao, Hongyu Lin, Xianpei Han, Le Sun, and Hua Wu. 2022. Unified structure generation for universal information extraction. *arXiv preprint arXiv:2203.12277*.
- Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, and Wei Lin. 2023a. [Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models](#).

- Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023b. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*.
- OpenAI. 2023. [Gpt-4 technical report](#).
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022a. [Training language models to follow instructions with human feedback](#).
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022b. [Training language models to follow instructions with human feedback](#).
- Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. [Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters](#). In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20*, page 3505–3506, New York, NY, USA. Association for Computing Machinery.
- Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Tali Bers, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M. Rush. 2022. [Multi-task prompted training enables zero-shot task generalization](#).
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. [Hugging-gpt: Solving ai tasks with chatgpt and its friends in hugging face](#).
- Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble, Chris Kelly, Nathaneal Scharli, Aakanksha Chowdhery, Philip Mansfield, Blaise Agüera y Arcas, Dale Webster, Greg S. Corrado, Yossi Matias, Katherine Chou, Juraj Gottweis, Nenad Tomasev, Yun Liu, Alvin Rajkomar, Joelle Barral, Christopher Semturs, Alan Karthikesalingam, and Vivek Natarajan. 2022. [Large language models encode clinical knowledge](#).
- Xiaoshuai Song, Keqing He, Pei Wang, Guanting Dong, Yutao Mou, Jingang Wang, Yunsen Xian, Xunliang Cai, and Weiran Xu. 2023. [Large language models meet open-world intent discovery and recognition: An evaluation of chatgpt](#).
- Xiaoshuai Song, Zhengyang Wang, Keqing He, Guanting Dong, Yutao Mou, Jinxu Zhao, and Weiran Xu. 2024. [Knowledge editing on black-box large language models](#).
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca.
- Erik F. Tjong Kim Sang and Fien De Meulder. 2003. [Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition](#). In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. [Llama: Open and efficient foundation language models](#).
- Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-sne. *Journal of machine learning research*, 9(11).
- Guan Wang, Sijie Cheng, Xianyu Zhan, Xiangang Li, Sen Song, and Yang Liu. 2023a. [Openchat: Advancing open-source language models with mixed-quality data](#).
- Pei Wang, Yejie Wang, Muxi Diao, Keqing He, Guanting Dong, and Weiran Xu. 2024a. [Multi-perspective consistency enhances confidence estimation in large language models](#).
- Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, and Guoyin Wang. 2023b. [Gpt-ner: Named entity recognition via large language models](#).
- Yejie Wang, Keqing He, Guanting Dong, Pei Wang, Weihao Zeng, Muxi Diao, Yutao Mou, Mengdi Zhang, Jingang Wang, Xunliang Cai, and Weiran Xu. 2024b. [Dolphocoder: Echo-locating code large language models with diverse and multi-objective instruction tuning](#).
- Yizhong Wang, Hamish Ivison, Pradeep Dasigi, Jack Hessel, Tushar Khot, Khyathi Raghavi Chandu, David Wadden, Kelsey MacMillan, Noah A Smith, Iz Beltagy, et al. 2023c. How far can camels go? exploring the state of instruction tuning on open resources. *arXiv preprint arXiv:2306.04751*.

- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023d. [Self-instruct: Aligning language models with self-generated instructions](#).
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.
- Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2023. [Magicoder: Source code is all you need](#).
- Shaohua Wu, Xudong Zhao, Tong Yu, Rongguo Zhang, Chong Shen, Hongli Liu, Feng Li, Hong Zhu, Jiangang Luo, Liang Xu, et al. 2021. Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning. *arXiv preprint arXiv:2110.04725*.
- Yuxiang Wu, Guanting Dong, and Weiran Xu. 2023. [Semantic parsing by large language models for intricate updating strategies of zero-shot dialogue state tracking](#).
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023a. [Wizardlm: Empowering large language models to follow complex instructions](#).
- Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, and Enhong Chen. 2023b. [Large language models for generative information extraction: A survey](#).
- Runxin Xu, Fuli Luo, Zhiyuan Zhang, Chuanqi Tan, Baobao Chang, Songfang Huang, and Fei Huang. 2021. [Raise a child in large language model: Towards effective and generalizable fine-tuning](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9514–9528, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Mingfeng Xue, Dayiheng Liu, Kexin Yang, Guanting Dong, Wenqiang Lei, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2023. [Occuquest: Mitigating occupational bias for inclusive large language models](#).
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. [Tree of thoughts: Deliberate problem solving with large language models](#).
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. [React: Synergizing reasoning and acting in language models](#).
- Wen-tau Yih, Matthew Richardson, Chris Meek, Ming-Wei Chang, and Jina Suh. 2016. [The value of semantic parse labeling for knowledge base question answering](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 201–206, Berlin, Germany. Association for Computational Linguistics.
- Shangjian Yin, Peijie Huang, Yuhong Xu, Haojing Huang, and Jiatian Chen. 2024. [Do large language model understand multi-intent spoken language ?](#)
- Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024. [Meta-math: Bootstrap your own mathematical questions for large language models](#).
- Hongyi Yuan, Zheng Yuan, Chuanqi Tan, Fei Huang, and Songfang Huang. 2023a. [HyPe: Better pre-trained language model fine-tuning with hidden representation perturbation](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3246–3264, Toronto, Canada. Association for Computational Linguistics.
- Lifan Yuan, Yangyi Chen, Ganqu Cui, Hongcheng Gao, Fangyuan Zou, Xingyi Cheng, Heng Ji, Zhiyuan Liu, and Maosong Sun. 2024. Revisiting out-of-distribution robustness in nlp: Benchmarks, analysis, and llms evaluations. *Advances in Neural Information Processing Systems*, 36.
- Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Chuanqi Tan, and Chang Zhou. 2023b. [Scaling relationship on learning mathematical reasoning with large language models](#).
- Zheng Yuan, Hongyi Yuan, Chuanqi Tan, Wei Wang, Songfang Huang, and Fei Huang. 2023c. [Rrhf: Rank responses to align language models with human feedback without tears](#).
- Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhua Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning. *arXiv preprint arXiv:2309.05653*.
- Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhua Chen. 2024. [Mammoth2: Scaling instructions from the web](#).
- Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, and Guoyin Wang. 2024a. [Instruction tuning for large language models: A survey](#).
- Zhen Zhang, Yuhua Zhao, Hang Gao, and Mengting Hu. 2024b. [Linkner: Linking local named entity recognition models to large language models using uncertainty](#).
- Gang Zhao, Xiaocheng Gong, Xinjie Yang, Guanting Dong, Shudong Lu, and Si Li. 2023. [Demosg: Demonstration-enhanced schema-guided generation for low-resource event extraction](#).

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. [Judging llm-as-a-judge with mt-bench and chatbot arena](#).

Chunting Zhou, Pengfei Liu, Puxin Xu, Srinu Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, et al. 2023a. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023b. [Instruction-following evaluation for large language models](#).

Sizhe Zhou, Yu Meng, Bowen Jin, and Jiawei Han. 2024. Grasping the essentials: Tailoring large language models for zero-shot relation extraction. *arXiv preprint arXiv:2402.11142*.

Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zhicheng Dou, and Ji-Rong Wen. 2024. [Large language models for information retrieval: A survey](#).

A SFT Datasets

We investigate the data composition issues of mathematical reasoning, coding, and general capabilities in the SFT stage from the following SFT datasets.

- **Code Alpaca** ([Chaudhary, 2023](#)) aims to build and share an instruction-following LLaMA model for code generation. which is fully based on Stanford Alpaca and contains 20K data used for fine-tuning the model. The Code Alpaca dataset has been open-sourced⁴.
- **GSM8K RFT** ([Yuan et al., 2023b](#)) is a mathematical dataset enhanced by integrating multiple reasoning paths based on the original GSM8K dataset ([Cobbe et al., 2021](#)) through the rejection sampling. It contains 7.5K questions and 110K responses in the training set. The GSM8k RFT dataset has been open-sourced⁵.
- **ShareGPT** refers to the multi-turn chatting histories used by Vicuna ([Chiang et al., 2023](#)). ShareGPT includes 86K human queries and responses from ChatGPT and other chatbots. The GSM8k RFT dataset has been open-sourced⁶.

The following table 2 presents the statistics of three datasets at different subset proportion (k).

Data Ratio	GSM8K RFT	CodeAlpaca	ShareGPT
K=1/1	110142	20022	86060
K=1/4	27535	5005	21515
K=1/16	6883	1251	5378
K=1/64	1720	312	1344
K=1/256	430	78	336

Table 2: Data statistics of three datasets at different subset proportion (k).

B Evaluation metrics

We use the following metrics to measure the aligned large language models.

⁴<https://github.com/sahil280114/codealpaca>

⁵<https://github.com/OFA-Sys/gsm8k-ScRel>

⁶Exact dataset of ShareGPT (<https://sharegpt.com/>) has not been released. We instead use a reproduced version from https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered cleaned raw dataset, and follow Vicuna preprocess.

- **HumanEval** (Chen et al., 2021) consists of 164 original programming problems, with an average of 9.6 test cases allocated to each problem. To ensure a thorough assessment of the functional correctness of LLM-synthesized code, HumanEval+ extends the number of test cases significantly, averaging at 774.8 test cases per problem. We use the same method as (Chen et al., 2021) to obtain unbiased estimates of Pass@k under greedy decoding. To facilitate the reproducibility of our results, we use the open-source github repository BigCode (Ben Allal et al., 2022) to evaluate all the HumanEval scores in this paper ⁷.
- **GSM8K** (Cobbe et al., 2021) is a math word problem dataset used to measure large language model math reasoning ability. We use the default test set to measure the model. We calculate the score based on greedy decoding accuracy (maj@1). In this paper, we use the open-source github repository gsm8k-ScRel ⁸ to evaluate all the GSM8k scores.
- **MT-Bench** (Zheng et al., 2023) is a significant benchmark that contribute to the evaluation and advancement of chatbot models and LLMs in different contexts. MT-Bench⁹ evaluates LLMs on multi-turn dialogues using comprehensive questions tailored to handling conversations. It provides a comprehensive set of questions specifically designed for assessing the capabilities of models in handling multi-turn dialogues.

We also supplement more benchmark evaluation results in the appendix F to verify the generalization of our conclusions:

- **MATH** (Hendrycks et al., 2021) is a dataset with challenging high-school math problems. Problems are classified into the following topics: Prealgebra, Algebra, Number Theory, Counting and Probability, Geometry, Intermediate Algebra, and Precalculus. Problems in MATH are harder and more diverse than in GSM8K. In this paper, we use the open-source github repository gsm8k-ScRel to evaluate all

the MATH scores. We use 500 test problems from (Lightman et al., 2023) as out-of-domain math benchmark.

- **MBPP** (Austin et al., 2021) consists of around 1,000 crowd-sourced Python programming problems, designed to be solvable by entry-level programmers, covering programming fundamentals, standard library functionality, and so on. Each problem consists of a task description, code solution and 3 automated test cases. To facilitate the reproducibility of our results, we use the open-source github repository BigCode (Ben Allal et al., 2022) to evaluate all the MBPP scores in this paper.

C Implementation Details

We fine-tune all the SFT datasets with 3 epochs and a batch size of 128 on NVIDIA A100 GPUs. We use 8 GPUs for 7B and 13B models, 16 GPUs for 33B models during fine-tuning. We use a peak learning rate of 2e-5 with a 3% learning rate warmup. We evaluate the results on the final epoch. We use greedy decode to calculate Pass@1 and maj@1. Since the scores of MT-bench will fluctuate, we conducted three experiments and took the average.

All experiments are conducted using the default template of the FastChat framework (Zheng et al., 2023), as shown in the figure below:

Prompt Template

A chat between a curious user and an artificial intelligence assistant. The assistant gives helpful, detailed, and polite answers to the user’s questions. **USER:** {Query}
ASSISTANT:

To facilitate the replication of our results, all datasets and evaluation benchmarks used in our experiments have been open-sourced and their detailed sources are indicated. We will also open-source our code after the blind review process.

D Estimating FLOPs of SFT

Training FLOPs. We mainly follow the notations of (Kaplan et al., 2020) here.

For each input sample of length n_{ctx} in SFT dataset (GSM8K, CodeAlpaca, ShareGPT), we can split it into two parts:

$$n_{ctx} = n_Q + n_R \quad (1)$$

⁷<https://github.com/bigcode-project/bigcode-evaluation-harness>

⁸<https://github.com/OFA-Sys/gsm8k-ScRel>

⁹<https://huggingface.co/spaces/lmsys/mt-bench>

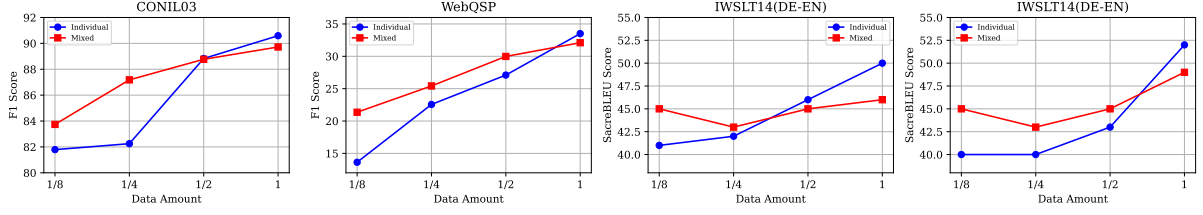


Figure 7: The scaling curve of LLaMA-7B in WebQSP, CoNLL 2003, IWSLT14(de-en), IWSLT14(de-en).

Model size	7B	13B	33B
<i>GSM8k RFT</i>			
SFT FLOPs	2.4×10^{18}	4.3×10^{18}	1.1×10^{19}
SFT GPI hrs	6.1	12.1	37.4
<i>Code Alpaca</i>			
SFT FLOPs	4.7×10^{17}	7.8×10^{17}	2.0×10^{18}
SFT GPI hrs	1.2	2.5	8.2
<i>ShareGPT</i>			
SFT FLOPs	2.2×10^{18}	3.9×10^{18}	9.7×10^{19}
SFT GPI hrs	5.4	10.9	34.0

Table 3: The statistics of FLOPs and GPU hours required for SFT. For 33B, we use DeepSpeed ZeRO3 (Rasley et al., 2020) for distributed training. All the GPU hours are based on NVIDIA A100 80GB GPU. Note we use non-embedding parameters to compute FLOPs in our experiments.

$$C_{\text{train}} \approx 6Nn_{\text{ctx}}N_s \quad (2)$$

where n_Q, n_R denotes the length of question and generated answers respectively. N, N_s denotes the non-embedding parameters and the numbers of samples.

Therefore, We estimate the SFT FLOPs following (Kaplan et al., 2020) and GPU times in Table 3.

E Validation Experiments in More SFT Abilities

To validate the generalization of our conclusions, we selected representative datasets to evaluate the capabilities of large models across different dimensions. These dimensions include **World Knowledge**: WebQuestionsSP (Yih et al., 2016), **Language Understanding**: CoNLL 2003 (Tjong Kim Sang and De Meulder, 2003), and **Translation**: IWSLT14 (Cettolo et al., 2014)

Experimental Design: Align the settings of RQ1 and RQ2, we introduce two settings as follows:

1. Individual Domain: We conduct SFT on LLaMA of various sizes using $\{1, 1/2, 1/4, 1/8\}$ proportions¹⁰ of the training set obtained from WebQSP, CoNLL 2003, and IWSLT14 separately. This allowed us to evaluate each ability with various data sizes and model sizes.

2. Mixed Domain: We sampled $\{1, 1/2, 1/4, 1/8\}$ amounts of training data from WebQSP, CoNLL 2003, and IWSLT14, and directly mixed them according to the corresponding proportions. In this way, we constructed datasets with fixed proportions of different ability domains, while varying the total data amount. These datasets are then used for fine-tuning the LLaMA models.

Analysis. As shown in Figure 7 and Table 4, we have following observations.

For the **individual domain**, the performance (P, R, F1) of the model in the language understanding (NER) task shows a positive correlation with the scaling curve of data volume. These two abilities exhibit similar scaling curve trends as the mathematical ability performance in RQ1. In the case of world knowledge (WebQSP), a similar positive correlation trend is observed in terms of F1 and Hits@1. However, when the data ratio is reduced from 1/4 to 1/8, there is a significant performance fluctuation, particularly in the performance of translation ability, which shows a relatively irregular trend. These conclusions further support the core conclusion of RQ1 that different data exhibit different scaling curves.

For the **mixed domains**, the findings align with the conclusions in RQ2, where abilities are improved with low-resource and decreased with high-resource compared to individual source abilities. This consistent conclusion holds for world knowledge, language understanding, and translation abilities.

¹⁰Because these three datasets have relatively small amounts of data (a few thousand), the scaling range is from 1/1 of the data volume to 1/8 of the data volume.

Datasets	CONIL03			WebQSP		IWSLT14	
	P	R	F1	F1	Hits@1	de-en	en-de
Single Domain(1/1)	91.89	89.33	90.59	33.51	64.12	50	52
Single Domain(1/2)	90.59	87.15	88.83	27.10	61.87	46	43
Single Domain(1/4)	85.24	79.46	82.25	22.56	61.38	42	40
Single Domain(1/8)	83.22	80.42	81.79	13.63	49.05	41	40
Mixed Domains(1/1)	91.74	87.79	89.72	32.10	63.70	46	49
Mixed Domains(1/2)	90.69	86.93	88.77	29.98	62.29	45	45
Mixed Domains(1/4)	88.81	85.62	87.18	25.42	58.02	43	43
Mixed Domains(1/8)	86.47	81.18	83.74	21.36	56.86	45	45

Table 4: Results in other domains for single and mixed source settings based on LLaMA-7B.

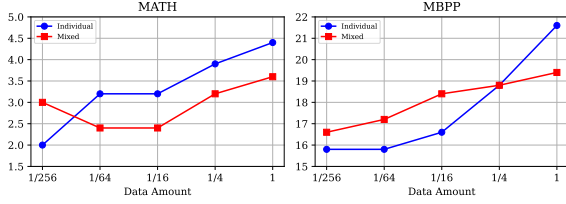


Figure 8: The scaling curve of LLaMA-7B on MATH and MBPP benchmarks.

F Results on OOD Benchmarks in Math and Code

To validate the generalization of our findings on other benchmarks, we utilized GSM8K and Code Alpaca as the training sets. We further evaluated the results on the individual domain, mixed domain, and different training strategies on other specialized ability benchmark, including MATH and MBPP, which is illustrated in Table 5 and Figure 8. We have the following findings:

(1) In the individual domain, LLaMA shows a positive correlation between performance in MATH and MBPP and the data volume (consistent with RQ1).

(2) Comparing the individual and mixed domains, LLaMA-7B exhibits a trade-off between high-resource performance conflict and low-resource performance gain in both MATH and MBPP (consistent with RQ2).

(3) Considering the general ability results shown in Table 1, we can observe that DMT maintains competitive results in MATH and MBPP while prioritizing general abilities. This further validates the effectiveness of DMT (consistent with RQ4).

G Visualization of Different Layers

In this section, we compared the visualization results of the baseline model of LLaMA-13B and DMT ($k=1/256$) in the starting layer (Layer1), mid-

dle layer (Layer15), and ending layer (Layer31) in Figure 9 and 10.

The visualization result of the starting layer are relatively chaotic, while the visualization results of the middle layer and the ending layer are clearer. And the results of the middle layer and the last layer are consistent in pointing out that both base model and model with DMT strategy exhibit a certain level of separation in the mathematical data representations, there remains a certain degree of overlap between the representations of code and general samples.

H Equal Data Amount VS. Equal Data Proportion

In a realistic SFT phrase for training general LLM, the data amount for different abilities is likely to differ. Therefore, instead of controlling the same amount of data, we select to mix datasets with the same proportion of subsets to better simulate real-world scenarios in above experiments. In addition, We further supplement the experimental results using different abilities mixed with the equal data amount and compare them with the results using the equal subset proportion in Table 6.

Equal Data amount Setting: we utilize the data amount of GSM8k RFT as the baseline. We sampled data with proportions of 1/16, 1/64, 1/256, and mixed samples of the same data amount from Code alpaca and ShareGPT.

Equal Proportion Setting: we sampled data with proportions of 1/16, 1/64, 1/256 according to the subset proportions of each dataset and mixed them, which is aligned with the setup in RQ2.

It can be observed that there is not a significant difference in the results of the three benchmark tests between the two settings. Therefore, these findings do not significantly impact the main experimental conclusions presented in the paper.

Methods	Math Benchmarks		Code Benchmarks	
	GSM8K	MATH	HumanEval	MBPP
<i>Individual domain (Scaling)</i>				
Single Domain(k=1/1)	49.10	4.4	18.4	21.6
Single Domain(k=1/4)	43.37	3.9	11.58	18.8
Single Domain(k=1/16)	35.90	3.2	12.19	16.6
Single Domain(k=1/64)	22.71	3.2	9.14	15.8
Single Domain(k=1/256)	12.7	2.0	5.48	15.8
<i>Mixed domain (Scaling)</i>				
Mixed Domain(k=1/1)	47.53	3.6	14.63	19.4
Mixed Domain(k=1/4)	41.98	3.2	9.14	18.8
Mixed Domain(k=1/16)	32.97	2.4	9.16	18.4
Mixed Domain(k=1/64)	25.77	2.4	14.63	17.2
Mixed Domain(k=1/256)	14.78	3.0	11.37	16.6
<i>Individual domain</i>				
General only	11.1	2.9	10.4	1.0
Math only	49.10	4.4	6.71	9.0
Code only	4.51	1.0	18.40	21.6
<i>Different Training Strategies</i>				
Multi-task learning	47.53	3.6	<u>14.63</u>	<u>19.4</u>
Sequential Training	31.39	2.0	15.85	15.8
Mixed Sequential Training	32.6	2.5	15.24	16.6
DMT (k=1/256)	<u>41.92</u>	3.6	17.68	19.8

Table 5: The detailed results of LLaMA-7B, 13B with different training strategies on OOD benchmarks.

Methods	GSM8K	HumanEval	MT-Bench
Mixed Domain(k=1/16, Equal Amount)	34.49	9.14	5.49
Mixed Domain(k=1/64, Equal Amount)	25.02	13.54	5.21
Mixed Domain(k=1/256, Equal Amount)	16.7	11.54	4.63
Mixed Domain(k=1/16, Equal Proportion)	32.97	9.16	5.52
Mixed Domain(k=1/64, Equal Proportion)	25.77	14.63	5.24
Mixed Domain(k=1/256, Equal Proportion)	14.78	11.37	4.41

Table 6: Comparative experiment between equal data amounts and equal subset proportions of different SFT abilities on LLaMA-7B

I Comparison Experiment of Different Training Sequences

To investigate the impact of training order on different SFT abilities, we have conducted additional experiments with six different training orders. The results and analysis of these experiments are provided in Table 7. Based on our findings, we conclude the following:

1. The SFT ability trained in the final stage tend to retain relatively good performance.
2. If general and code abilities are trained in the first two stages, there is a noticeable performance decrease in code capability, while math capability does not show significant impact. One possible reason is that the task format of code generation

and general ability exhibits similar data distributions (as discussed in RQ3 and Discussion1). This can result in a more severe catastrophic forgetting phenomenon during continuous fine-tuning.

J Detailed Results of experiments

J.1 Results of Different Random Seeds

For each dataset, we employed random selection by utilizing a random function with three distinct seeds for sampling. Subsequently, we conducted a comparative analysis of the results obtained from different subsets on the three benchmark tests. The specific details are presented in Table 8. It can be observed that DMT maintains its superiority under three different random seed settings. The influence

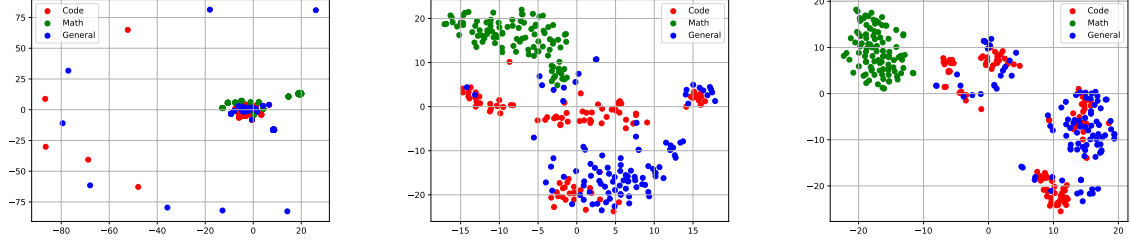


Figure 9: From left to right are the visualization results of starting layer (Layer1), middle layer (Layer15), and ending layer (Layer31) on LLaMA-7B.

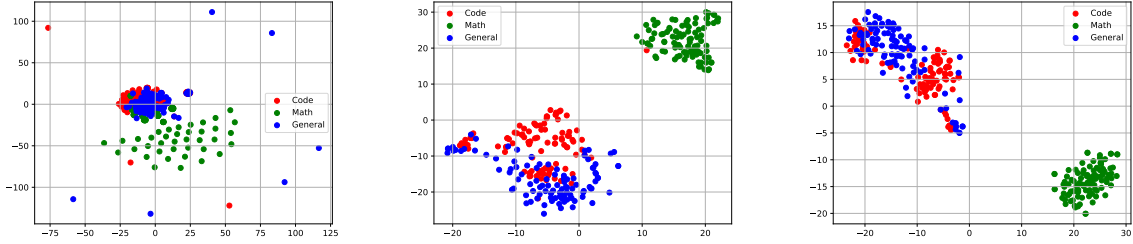


Figure 10: From left to right are the visualization results of starting layer (Layer1), middle layer (Layer15), and ending layer (Layer31) on LLaMA-7B with DMT($k=1/256$) strategy.

of different subsets on experimental results is not a key factor and does not affect the overall trend.

J.2 Results of Single Source and Mixed Source

In Table 9 and Table 10, we report the detailed comparative results between mix domains and individual domains for LLaMA-7B, 3B and 33B, as the supplemental results in RQ2.

J.3 Results of Data Ratio (k)

In Table 11, we report The detailed results of the data ratio (k) between specific abilities and general abilities on three benchmarks, as the supplemental results in RQ3.

J.4 Results of Specialized Data Amount of DMT

In Table 12, we report The detailed results of LLaMA-7B, 13B, 33B with different training strategies on three benchmarks, as the supplemental results in RQ4.

J.5 Results of MT-Bench

In Figure 11, we report detailed results of LLaMA-7B, 13B, 33B with different training strategies on MT-Bench, which include coding, extraction, humanities, math, reasoning, roleplay, stem and writing abilities.

J.6 Supplemental Results for Discussion

In Figure 12, we report the t-SNE visualizations of LLaMA-7B and LLaMA-7B with DMT($k=1/256$) strategy. What’s more, the bottom figure represents the scaling relationship of LLaMA-7B with DMT($k=1/256$) under different values of K .

Moreover, in Table 13, we report The detailed results of LLaMA-7B, 13B, 33B with different training strategies on three benchmarks, as the supplemental results in RQ4.

Methods	GSM8K	HumanEval	MT-Bench
Code $\rightarrow$ Math $\rightarrow$ General	31.39	15.85	5.72
Math $\rightarrow$ Code $\rightarrow$ General	29.71	15.85	5.65
Code $\rightarrow$ General $\rightarrow$ Math	48.21	9.75	4.7
General $\rightarrow$ Code $\rightarrow$ Math	48.21	7.9	4.59
General $\rightarrow$ Math $\rightarrow$ Code	37.60	15.85	3.79
Math $\rightarrow$ General $\rightarrow$ Code	26.45	16.46	3.68

Table 7: Results of different sequential training for LLaMA-7B

Methods	LLaMA -7B			LLaMA -13B			LLaMA -33B		
	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench
<i>Different Training Strategies</i>									
Multi-task learning	47.53	14.63	5.76	50.94	19.50	5.73	56.69	18.9	6.07
Sequential Training	31.39	15.85	5.72	39.12	<u>20.12</u>	5.93	47.27	<u>24.80</u>	6.73
Mixed Sequential Training	32.60	15.24	6.02	40.48	18.30	5.93	44.24	24.4	6.43
DMT(k=1/256,random seed=1)	41.92	<u>17.68</u>	<u>6.08</u>	<u>46.47</u>	19.50	6.03	<u>56.36</u>	25.00	6.73
DMT(k=1/256,random seed=2)	41.31	<u>17.68</u>	6.02	45.85	18.90	<u>6.08</u>	55.64	<u>24.80</u>	<u>6.71</u>
DMT(k=1/256,random seed=3)	<u>42.03</u>	18.21	6.13	46.22	20.52	6.10	56.12	25.30	6.73

Table 8: The results of LLaMA-7B, 13B, 33B under different training strategies on three benchmarks. We tested the results of DMT on randomly sampling k proportion of specified data under three random seeds.

Methods	LLaMA-7B			LLaMA-13B		
	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench
Single(k=1)	49.10	18.4	5.88	51.4	18.4	6.13
Single(k=1/4)	43.37	11.58	5.85	48.59	13.41	6.03
Single(k=1/16)	35.90	12.19	5.61	43.00	12.80	5.66
Single(k=1/64)	22.71	9.14	5.11	27.40	12.20	5.24
Single(k=1/256)	12.70	5.48	4.00	18.40	10.36	2.95
Mix(k=1)	47.53	14.63	5.76	50.49	17.10	5.73
Mix(k=1/4)	41.98	9.14	5.48	48.52	14.00	5.61
Mix(k=1/16)	32.97	9.16	5.22	40.63	14.60	5.52
Mix(k=1/64)	25.77	14.63	5.27	33.2	17.68	5.24
Mix(k=1/256)	14.78	11.37	4.11	24.94	12.19	4.4

Table 9: Comparative experiments between mix domains and individual domains for LLaMA-7B, 13B.

Methods	GSM8K	HumanEval	MT-Bench
Single(k=1)	57.91	26.82	6.63
Single(k=1/4)	56.10	25.61	6.66
Single(k=1/16)	54.60	21.95	6.17
Single(k=1/64)	44.60	18.59	5.99
Single(k=1/256)	29.21	14.02	2.3
Mix(k=1)	56.69	18.9	6.07
Mix(k=1/4)	54.54	22.56	5.92
Mix(k=1/16)	53.33	26.82	6.26
Mix(k=1/64)	46.66	18.6	5.73
Mix(k=1/256)	36.54	17.68	4.58

Table 10: Comparative experiments between mix domains and individual domains for LLaMA-33B.

Model size	GSM8K	HumanEval	MT-Bench
Mix[(code,math),1 general]	47.53	14.63	5.76
Mix[(code,math),1/4 general]	48.44	15.85	5.73
Mix[(code,math),1/16 general]	47.99	15.24	5.27
Mix[(code,math),1/64 general]	47.23	14.63	5.16
Mix[(code,math),1/256 general]	48.52	16.46	4.69
Mix[1(code,math),general]	47.53	14.63	5.76
Mix[1/4(code,math),general]	41.31	10.97	5.81
Mix[1/16(code,math),general]	33.20	11.58	5.76
Mix[1/64(code,math),general]	25.17	12.19	5.84
Mix[1/256(code,math),general]	16.52	9.14	5.82
Mix[1(code,math),1/64general]	47.68	14.63	5.09
Mix[1/4(code,math),1/64general]	43.29	12.19	5.07
Mix[1/16(code,math),1/64general]	33.81	12.19	5.17
Mix[1/64(code,math),1/64general]	26.23	12.19	5.12
Mix[1/256(code,math),1/64general]	18.27	10.36	5.12

Table 11: The detailed results of the data ratio (k) between specific abilities and general abilities on three benchmarks.

Methods	LLaMA-7B			LLaMA-13B		
	GSM8K	HumanEval	MT-Bench	GSM8K	HumanEval	MT-Bench
<i>Individual domain</i>						
General only	11.10	10.42	5.88	14.02	16.40	6.13
Math only	49.10	6.71	2.53	51.40	12.8	2.54
Code only	4.51	18.40	4.30	5.15	17.1	3.53
<i>Different Training Strategies</i>						
Multi-task learning	47.53	14.63	5.76	50.94	19.5	5.73
Sequential Training	31.39	15.85	5.72	39.12	20.12	5.93
Mixed Sequential Training	32.6	15.24	6.02	40.48	18.30	5.93
DMT (k=1)	45.79	14.02	5.63	50.49	16.46	5.76
DMT (k=1/4)	48.37	13.41	5.69	50.18	18.9	5.83
DMT (k=1/16)	43.3	15.24	5.78	48.59	18.9	5.96
DMT (k=1/64)	42.53	15.85	6.01	47.61	15.24	6.03
DMT (k=1/256)	41.92	17.68	6.08	46.47	19.5	6.03

Table 12: The detailed results of LLaMA-7B, 13B with different training strategies on three benchmarks.

Model size	GSM8K	HumanEval	MT-Bench
1/1 Mix(code,math,general(w/o code math))	49.05	17.68	5.80
1/4 Mix(code,math,general(w/o code math))	43.13	15.85	5.71
1/16 Mix(code,math,general(w/o code math))	36.23	10.36	5.38
1/64 Mix(code,math,general(w/o code math))	25.62	10.97	5.21
1/256 Mix(code,math,general(w/o code math))	15.31	11.37	4.38

Table 13: The scaling curve after ablating code and math-related samples from ShareGPT

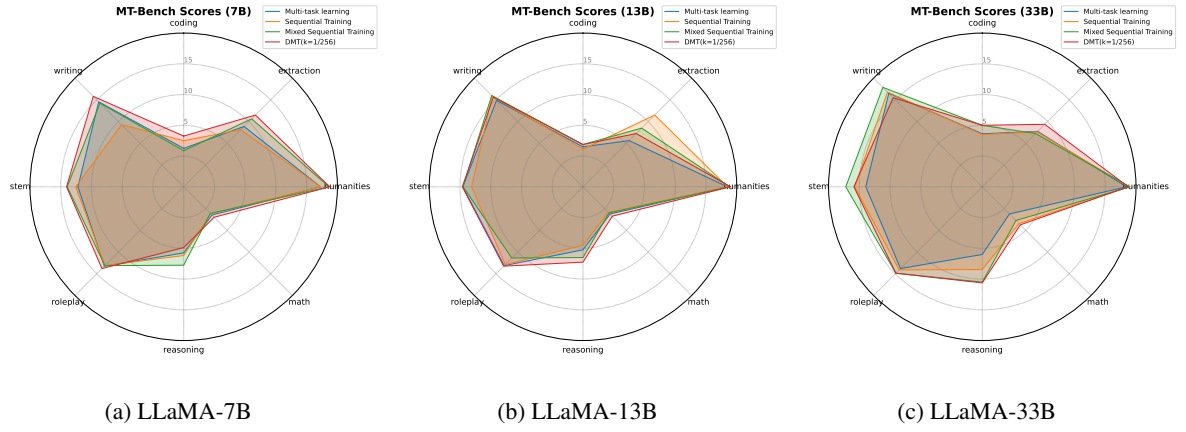


Figure 11: The detailed results of LLaMA-7B, 13B, 33B with different training strategies on MT-Bench.



Figure 12: Figures show the t-SNE visualizations of LLaMA-7B and LLaMA-7B with DMT(k=1/256) strategy.